claude-windows / 662b86ab-d2e3-4c02-8a0d-ec7a9071aba8

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\662b86ab-d2e3-4c02-8a0d-ec7a9071aba8\subagents\workflows\wf_87b22fa4-6ac\agent-af6509c7fb7fc4626.jsonl`
- SHA-256: `d10ec13ede9c402eaf990a03129c7c505778ed486d668229946e8f8c0c0f998c`
- Source modified: `2026-06-21T10:47:03+00:00`
- Imported at: `2026-07-05T16:48:27+00:00`
- project: `wf_87b22fa4-6ac`
- session_id: `662b86ab-d2e3-4c02-8a0d-ec7a9071aba8`

Transcript

1. user (2026-06-21T10:45:15.844Z)

Review the COMPUTE_DECOUPLED_SERVING **M6** change just committed on branch feat/serving-m6-cloudrun-dispatch (HEAD). Inspect with: git diff origin/master...HEAD , and read the full files:
- backend/compute/base.py (ComputeBackend ABC + ComputeJobSpec + ComputeResult)
- backend/compute/cloud_run.py (CloudRunCompute: dispatch a Cloud Run Job via injectable CloudRunJobClient, then bucket-poll a done-marker via execution_policy.poll_until + the storage layer; GoogleCloudRunJobClient = real client, lazy google-cloud-run)
- backend/compute/__init__.py (get_compute_backend(config) ? backend only for compute_target=cloud_run, else None = run inline)
- backend/worker/__main__.py (cmd_infer: dispatch branch for cloud_run + _dispatch_remote; --done-uri + _write_done_marker on success)
- deploy/cloudrun/Dockerfile + README.md (the L4 image)
- tests/test_compute_backend.py
CONTRACT: DEFAULT-OFF ? only compute_target=cloud_run dispatches; ""/local_gpu/cpu_mock run the worker's EXISTING in-process path BYTE-IDENTICALLY. The dispatched container must run INLINE (never re-dispatch) via forced compute_target=local_gpu+native. Completion = a bucket done-marker (not the Cloud Run API status), D16. The real client is owner-verified on M7 (CI uses a fake). Report ONLY real, grounded issues with file:line. Empty findings array if a lens finds nothing real ? do not invent nits.

LENS = CLOUD PROTOCOL ROBUSTNESS / RACE / FAILURE-MODES. (1) The bucket-poll: CloudRunCompute polls done_uri via storage.exists then reads it. Is there a read-after-write race (marker visible to exists() but the body not yet flushed ? _read_json gets partial JSON)? The worker writes the marker LAST (after pushing outputs) ? is the ORDER safe so the dispatcher never reads a marker before report.json exists? (2) Failure: a failed container writes NO marker ? poll_until hits max_wait_sec ? PolicyTimeout propagates out of run_infer ? out of cmd_infer UNCAUGHT (traceback / nonzero). Is that acceptable, or should _dispatch_remote catch PolicyTimeout and return a clean rc? (3) _read_marker returns {} when the marker is empty/garbage but present ? poll terminates with empty dict ? video_id="" ? outputs_uri="" ? report skipped. Is an empty-but-present marker handled sanely? (4) The real GoogleCloudRunJobClient.run_job ? is the google-cloud-run v2 usage plausible (RunJobRequest/Overrides/ContainerOverride/JobsClient)? Flag if clearly wrong (owner verifies on M7 but obvious API errors are worth noting). Find real robustness gaps.

2. assistant (2026-06-21T10:45:18.429Z)

I'll review the M6 change on the compute-decoupled serving track. Let me start by pulling the latest and inspecting the diff and files.

3. user (2026-06-21T10:45:22.443Z)

feat/serving-m6-cloudrun-dispatch
2266e91 feat(serving): M6 ? Cloud Run GPU dispatch shim + worker image (default-OFF, mocked)
---DIFF STAT---

```
 backend/compute/__init__.py   |  61 ++++++++++++
 backend/compute/base.py       |  52 ++++++++++
 backend/compute/cloud_run.py  | 141 ++++++++++++++++++++++++++++
 backend/worker/__main__.py    |  51 ++++++++++
 deploy/cloudrun/Dockerfile    |  61 ++++++++++++
 deploy/cloudrun/README.md     |  59 ++++++++++++
 tests/test_compute_backend.py | 222 ++++++++++++++++++++++++++++++++++++++++++
 7 files changed, 647 insertions(+)
```

4. assistant (2026-06-21T10:45:24.391Z)

Let me read all the files in parallel.

5. user (2026-06-21T10:45:25.342Z)

```
1       """The ``ComputeBackend`` ABC ? one interface for *where* the rally core runs.
2
3       COMPUTE_DECOUPLED_SERVING M6. Mirrors the ``StorageBackend`` ABC+factory idiom
4       (``backend/storage/``): a ``ComputeBackend`` decides **where** the pure
DETECT?WINDOW?STITCH
5       core executes ? in-process on the local box (truly-local) or on a **Cloud Run GPU Job**
6       (cloud-serving) ? without the core ever branching on the profile (D1).
7
8       The seam is **default-OFF**: only ``deployment.compute_target == "cloud_run"`` selects a
9       non-local backend. Every other target (``""`` / ``local_gpu`` / ``cpu_mock``) keeps the
worker's
10      existing in-process path **byte-identical** ? the factory returns ``None`` (= "run
inline").
11      """
12
13      from __future__ import annotations
14
15      from abc import ABC, abstractmethod
16      from dataclasses import dataclass, field
17      from typing import Optional
18
19
20      @dataclass(frozen=True)
21      class ComputeJobSpec:
22          """One inference job, profile-agnostic. ``video_id`` is the MD5 the worker will
(re)compute
23          from the SAME input bytes, so the output always lands at
``<out_uri>/outputs/<video_id>/`` ?
24          deterministic regardless of where the job ran."""
25
26          video_uri: str                          # input (gs://? for cloud; a path/local://
for local)
27          out_uri: str                            # output ROOT (gs://? bucket for cloud)
28          segmenter: Optional[str] = None
29          # Raw worker ``--set k=v`` overrides. A tuple (not list/dict) so the spec stays
frozen/hashable.
30          config_overrides: tuple[str, ...] = ()
31
32
33      @dataclass(frozen=True)
34      class ComputeResult:
35          """The terminal state of a dispatched job. ``returncode`` mirrors the worker's exit
code
36          (0 ok / 2 validation-or-config / 3 segmenter-failure). ``video_id`` is known once
the job's
37          done-marker is read; ``outputs_uri`` is where ``intervals.csv`` / ``report.json``
landed."""
38
39          returncode: int
40          outputs_uri: str = ""
```

```
41          video_id: str = ""
42          report: dict = field(default_factory=dict)
43
44
45      class ComputeBackend(ABC):
46          """Backend-agnostic compute. ``run_infer`` dispatches ONE inference job and returns
only once
47          its outputs are durably available (bucket-poll for cloud), so a caller treats local
and cloud
48          execution identically."""
49
50          @abstractmethod
51          def run_infer(self, spec: ComputeJobSpec) -> ComputeResult:
52              """Run ``spec`` to completion and return its ``ComputeResult``."""
53
```

6. user (2026-06-21T10:45:25.833Z)

```
1       """``CloudRunCompute`` ? dispatch one inference job to a Cloud Run GPU **Job** and
bucket-poll
2       for its completion (COMPUTE_DECOUPLED_SERVING M6, decisions D16 + D7).
3
4       Flow (the cloud-serving compute path):
5         1. **Dispatch** a Cloud Run Job execution whose container runs ``python -m
backend.worker infer
6            --video-uri <gs> --out-uri <gs> --done-uri <gs> ?`` (the SAME worker, on the L4
GPU).
7         2. **Bucket-poll** ``done_uri`` via ``execution_policy.poll_until`` + the storage
layer (D16 ? the
8            completion signal is a tiny marker object the worker writes, NOT the Cloud Run API
status, so
9            it is storage-backed and reuses the existing park/resume primitives). Scale-to-zero
(D7): the
10           cold-start is absorbed by the async poll.
11        3. **Read** the marker (it carries ``video_id`` + the worker's returncode) ? fetch
12           ``outputs/<video_id>/report.json`` ? ``ComputeResult``.
13
14      The dispatched container is forced to run **INLINE** (``compute_target=local_gpu`` +
native detector)
15      so it never recursively re-dispatches ? the dispatcher decides cloud, the container just
computes.
16
17      The Cloud Run trigger is an **injected client** (``CloudRunJobClient``) so the whole
shim is unit-tested
18      with a fake (no GCP). The real ``GoogleCloudRunJobClient`` lazy-imports ``google-cloud-
run`` and is
19      owner-verified on the M7 real run.
20      """
21
22      from __future__ import annotations
23
24      import json
25      import logging
26      import uuid
27      from abc import ABC, abstractmethod
28      from typing import Any, List, Optional
29
30      from backend.compute.base import ComputeBackend, ComputeJobSpec, ComputeResult
31      from backend.infrastructure.execution_policy import DEFAULT_POLICY, ExecutionPolicy,
poll_until
32      from backend.storage import fetch, parse_uri
33      from backend.storage import get_storage
34
35      LOG = logging.getLogger("compute.cloud_run")
36
```

```
    # The container worker must run INLINE on the GPU ? never re-enter the cloud-dispatch
branch.
    # (compute_target=local_gpu disarms the cmd_infer dispatch guard; native = the L4 WASB
runner.)
    _DEFAULT_CONTAINER_OVERRIDES: tuple[str, ...] = (
        "deployment.compute_target=local_gpu",
        "indexing.detector_impl=native",
    )


    class CloudRunJobClient(ABC):
        """Triggers a Cloud Run **Job** execution with per-execution container arg
overrides.

        Abstracted so ``CloudRunCompute`` is testable with a fake (no GCP). A real
implementation
        overrides the job's container ``args`` for this execution and starts it."""

        @abstractmethod
        def run_job(self, job_name: str, argv: List[str], *, region: str,
                    project: Optional[str] = None) -> str:
            """Start a Cloud Run Job execution running ``python -m backend.worker <argv>``;
return an
            execution id/name (for logs). Does NOT block on completion (the caller bucket-
polls)."""


    class GoogleCloudRunJobClient(CloudRunJobClient):
        """Real client ? lazy-imports ``google-cloud-run``. Owner-verified on the M7 real
run; CI uses
        a fake, so this code path is never exercised in tests (the SDK isn't a test
dependency)."""

        def run_job(self, job_name: str, argv: List[str], *, region: str,
                    project: Optional[str] = None) -> str:
            try:
                from google.cloud import run_v2  # type: ignore[attr-defined]
            except Exception as exc:  # pragma: no cover - real SDK not present in CI
                raise RuntimeError(
                    "google-cloud-run is required for CloudRunCompute's real client; install
it on the "
                    "control plane (it is intentionally NOT a CI/test dependency)."
                ) from exc
            client = run_v2.JobsClient()  # pragma: no cover - exercised only on the real M7
run
            name = client.job_path(project, region, job_name)
            overrides = run_v2.RunJobRequest.Overrides(

container_overrides=[run_v2.RunJobRequest.Overrides.ContainerOverride(args=argv)]
            )
            op = client.run_job(request=run_v2.RunJobRequest(name=name,
overrides=overrides))
            return getattr(op, "metadata", None) and getattr(op.metadata, "name", "") or
"execution"


    class CloudRunCompute(ComputeBackend):
        """Dispatch to a Cloud Run GPU Job + bucket-poll for completion."""

        def __init__(self, *, run_client: CloudRunJobClient, job_name: str, region: str,
                     project: Optional[str] = None, storage_config: Optional[dict] = None,
                     policy: ExecutionPolicy = DEFAULT_POLICY,
                     token: Optional[str] = None,
                     container_overrides: Optional[tuple[str, ...]] = None) -> None:
            self._client = run_client
```

```
89              self._job_name = job_name
90              self._region = region
91              self._project = project
92              self._storage_config = storage_config or {}
93              self._policy = policy
94              # The dispatch token keys the done-marker so concurrent jobs to one bucket don't
collide.
95              # None ? a fresh uuid4 per run_infer call (prod); injected ? deterministic
(tests).
96              self._token = token
97              self._container_overrides = (
98                  _DEFAULT_CONTAINER_OVERRIDES if container_overrides is None else
tuple(container_overrides)
99              )
100
101         def run_infer(self, spec: ComputeJobSpec) -> ComputeResult:
102             out_root = spec.out_uri.rstrip("/")
103             token = self._token or uuid.uuid4().hex
104             done_uri = f"{out_root}/_dispatch/{token}.done"
105             argv: List[str] = ["infer", "--video-uri", spec.video_uri, "--out-uri",
spec.out_uri,
106                                "--done-uri", done_uri]
107             if spec.segmenter:
108                 argv += ["--segmenter", spec.segmenter]
109             for kv in (*spec.config_overrides, *self._container_overrides):
110                 argv += ["--set", kv]
111
112             execution = self._client.run_job(self._job_name, argv, region=self._region,
113                                              project=self._project)
114             LOG.info("cloud-run: dispatched job=%s exec=%s; bucket-polling %s",
115                      self._job_name, execution, done_uri)
116
117             marker = poll_until(lambda: self._read_marker(done_uri), self._policy,
118                                 label="cloud-run-infer", logger=LOG)
119             video_id = str(marker.get("video_id", ""))
120             outputs_uri = f"{out_root}/outputs/{video_id}/" if video_id else ""
121             report = self._read_json(f"{outputs_uri}report.json") if outputs_uri else {}
122             rc = int(marker.get("returncode", 0))
123             LOG.info("cloud-run: job=%s done (video_id=%s rc=%d)", self._job_name, video_id,
rc)
124             return ComputeResult(returncode=rc, outputs_uri=outputs_uri, video_id=video_id,
report=report)
125
126         # --- storage-backed completion signal (bucket-poll) ---
127         def _read_marker(self, done_uri: str) -> Optional[dict]:
128             """The done-marker dict if the worker has written it, else None ? poll_until
keeps waiting."""
129             ref = parse_uri(done_uri)
130             if not get_storage(ref.backend, config=self._storage_config).exists(ref):
131                 return None
132             return self._read_json(done_uri) or {}  # empty dict still terminates the poll
(present == done)
133
134         def _read_json(self, uri: str) -> dict:
135             try:
136                 local = fetch(uri, config=self._storage_config)
137                 with open(local, "r", encoding="utf-8") as f:
138                     data: Any = json.load(f)
139                 return data if isinstance(data, dict) else {}
140             except Exception:
141                 return {}
142
```

7. user (2026-06-21T10:45:26.295Z)

```
"""Pluggable compute layer: in-process (default) + a Cloud Run GPU Job dispatch shim.

COMPUTE_DECOUPLED_SERVING M6. ``get_compute_backend(config)`` is the seam: it returns a
``ComputeBackend`` ONLY when a non-local ``deployment.compute_target`` is selected, and
``None``
otherwise ? ``None`` means "run the worker's existing in-process path" (today's
behaviour, byte
identical). So the seam is **default-OFF**: with no profile / ``local_gpu`` /
``cpu_mock`` nothing
changes. Mirrors ``backend/storage``'s lazy factory idiom (cloud SDKs imported only when
needed).
"""

from __future__ import annotations

import os
from typing import Any, Optional

from backend.compute.base import ComputeBackend, ComputeJobSpec, ComputeResult
from backend.infrastructure.execution_policy import DEFAULT_POLICY, ExecutionPolicy

__all__ = [
    "ComputeBackend", "ComputeJobSpec", "ComputeResult", "get_compute_backend",
]


def _storage_dict(config: Any) -> dict:
    storage = getattr(config, "storage", None)
    if storage is None:
        return {}
    if hasattr(storage, "model_dump"):
        return storage.model_dump()  # type: ignore[no-any-return]
    return dict(storage) if isinstance(storage, dict) else {}


def get_compute_backend(
    config: Any,
    *,
    run_client: Optional[Any] = None,
    policy: ExecutionPolicy = DEFAULT_POLICY,
    token: Optional[str] = None,
) -> Optional[ComputeBackend]:
    """Return the compute backend for ``config.deployment.compute_target``, or ``None``
to run
    in-process (the default-OFF path).

    Only ``compute_target == "cloud_run"`` returns a non-None backend (a
``CloudRunCompute``). Its
    Cloud Run coordinates come from the control-plane env (``CLOUD_RUN_JOB`` /
``CLOUD_RUN_REGION``
    / ``GOOGLE_CLOUD_PROJECT``) so no secrets live in config. ``run_client`` is
injectable (tests
    pass a fake; production defaults to the real ``google-cloud-run`` client, owner-
verified on M7).
    """
    target = getattr(getattr(config, "deployment", None), "compute_target", "") or ""
    if target != "cloud_run":
        return None  # local_gpu / cpu_mock / "" ? run inline (today's path, unchanged)

    # Lazy import so the cloud SDK shim is only pulled when a cloud target is actually
selected.
    from backend.compute.cloud_run import CloudRunCompute, GoogleCloudRunJobClient

    job_name = os.environ.get("CLOUD_RUN_JOB", "rally-worker")
    region = os.environ.get("CLOUD_RUN_REGION", "asia-south1")
```

```
56          project = os.environ.get("GOOGLE_CLOUD_PROJECT")
57          client = run_client if run_client is not None else GoogleCloudRunJobClient()
58          return CloudRunCompute(
59              run_client=client, job_name=job_name, region=region, project=project,
60              storage_config=_storage_dict(config), policy=policy, token=token,
61          )
62
```

8. user (2026-06-21T10:45:27.313Z)

```
1       """Worker dispatcher ? pull from storage ? run the pipeline ? push outputs back.
2
3           python -m backend.worker infer --video-uri <uri> --out-uri <prefix> \
4               [--segmenter wasb_hybrid] [--config config.json] [--set k=v ...] [--scratch DIR]
5           python -m backend.worker train ...        # Gen-0 train-from-bucket (follow-up)
6
7       The URI scheme selects the storage backend via ``backend.storage`` (a bare path /
``local://``
8       is an identity passthrough; ``s3://`` / ``gcs://`` is an object store), so existing
local runs
9       are byte-for-byte unchanged. Combine with ``RALLY_DETECTOR_IMPL=stub`` and
10      ``--set indexing.skip_ai_handoff=true`` for a **secret-free, GPU-free** run (no Gemini
key) ?
11      the path validated end-to-end on DigitalOcean (``docs/SETUP_NEW_MACHINE.md``).
12
13      This is a thin orchestrator: it composes the SAME building blocks the ``backend.main``
CLI uses
14      (validator registry ? segmenter ? report), never a forked pipeline.
15      """
16      from __future__ import annotations
17
18      import argparse
19      import csv
20      import json
21      import logging
22      import os
23      import sys
24      import tempfile
25      from typing import Any, List
26
27      from backend import storage
28      from backend.compute import ComputeJobSpec, get_compute_backend
29      from backend.config import (
30          StartupCheckError,
31          enforce_startup_checks,
32          load_config,
33          probe_cuda_available,
34      )
35      from backend.infrastructure.database import Database
36      from backend.pipeline.segmenters.base import segmenter_registry
37      from backend.pipeline.validators.base import registry as validator_registry
38      from backend.reporting import emit_indexer_report
39      from backend.results import Failure
40      from backend.utils.hashing import compute_video_id
41
42      logger = logging.getLogger("backend.worker")
43
44
45      def _coerce(v: str) -> Any:
46          """Coerce a ``--set`` string value to bool/int/float where it round-trips, else
str."""
47          low = v.lower()
48          if low in ("true", "false"):
49              return low == "true"
50          for cast in (int, float):
```

```
            try:
                return cast(v)
            except ValueError:
                pass
        return v


    def _apply_overrides(config: Any, sets: List[str]) -> None:
        """Apply ``a.b.c=value`` overrides onto the typed config (e.g.
indexing.skip_ai_handoff=true)."""
        for kv in sets or []:
            key, sep, val = kv.partition("=")
            if not sep:
                raise ValueError(f"--set expects k=v, got {kv!r}")
            parts = key.split(".")
            obj = config
            for p in parts[:-1]:
                obj = getattr(obj, p)
            setattr(obj, parts[-1], _coerce(val))


    def _write_intervals_csv(segments: List[dict], path: str) -> None:
        """Decimal-second ``[start,end)`` rallies + shot count / ending reason ? the join-
friendly
        artifact (same schema as the rally-annotator labels)."""
        with open(path, "w", newline="") as f:
            w = csv.writer(f)
            w.writerow(["start", "end", "shot_count", "ending_reason"])
            for s in segments:
                w.writerow([
                    f'{float(s.get("start_time", 0.0)):.3f}',
                    f'{float(s.get("end_time", 0.0)):.3f}',
                    s.get("shot_count", ""),
                    s.get("ending_reason", s.get("shot_count_confidence", "")),
                ])


    def _write_report_json(video_id: str, segments: List[dict], status: str, path: str) ->
None:
        with open(path, "w") as f:
            json.dump({
                "video_id": video_id,
                "status": status,
                "segment_count": len(segments),
                "segments": [{"start": float(s.get("start_time", 0.0)),
                              "end": float(s.get("end_time", 0.0))} for s in segments],
            }, f, indent=2)


    def _run_startup_checks(config: Any) -> bool:
        """M5 per-profile startup checks for the worker (the compute process, so it probes
CUDA
        best-effort). Logs warnings; returns False on a FATAL coherence error so the caller
aborts
        cleanly (rc 2) rather than run an expensive job under an incoherent profile. A true
no-op
        (returns True, ZERO work) when no deployment.profile is selected.

        The CUDA probe is gated behind an active profile: ``probe_cuda_available()`` lazily
imports
        torch, and pre-M5 the worker's infer/train path was torch-free (detection is
delegated to a
        WSL/native subprocess). Probing only when a profile is set keeps the default-OFF
path a true
        no-op of work, not just of output."""
```

```
        profile = getattr(getattr(config, "deployment", None), "profile", "") or ""
        cuda = probe_cuda_available() if profile else None  # torch-free on the default-OFF
path
        try:
            enforce_startup_checks(config, cuda_available=cuda, logger=logger)
            return True
        except StartupCheckError:
            return False  # already logged at ERROR by enforce_startup_checks


    def _dispatch_remote(compute: Any, args: argparse.Namespace) -> int:
        """M6: hand ONE infer job to a remote ComputeBackend (Cloud Run) and return its rc
once the
        outputs are durably in the bucket (the backend bucket-polls the done-marker). The
dispatched
        container runs this SAME cmd_infer INLINE (compute_target forced local_gpu) ? never
re-dispatch."""
        spec = ComputeJobSpec(
            video_uri=args.video_uri, out_uri=args.out_uri,
            segmenter=args.segmenter, config_overrides=tuple(args.set or ()),
        )
        result = compute.run_infer(spec)
        logger.info("[worker] remote compute done: video_id=%s rc=%d outputs=%s",
                    result.video_id, result.returncode, result.outputs_uri)
        return result.returncode


    def _write_done_marker(done_uri: str, video_id: str, returncode: int, segment_count:
int,
                           status: str, storage_cfg: dict, scratch: str) -> None:
        """M6: write a tiny JSON completion marker to ``done_uri`` (via the storage layer)
for a remote
        dispatcher's bucket-poll. Best-effort ? never raises into the worker's success
return."""
        try:
            marker = {"video_id": video_id, "returncode": returncode,
                      "segment_count": segment_count, "status": status}
            local = os.path.join(scratch, "_done.json")
            with open(local, "w", encoding="utf-8") as f:
                json.dump(marker, f)
            storage.put(local, done_uri, config=storage_cfg)
            logger.info("[worker] wrote completion marker -> %s", done_uri)
        except Exception as e:  # never fail a good run because the marker push hiccuped
            logger.warning("[worker] could not write done-marker %s: %s", done_uri, e)


    def cmd_infer(args: argparse.Namespace) -> int:
        config = load_config(args.config) if args.config else load_config()
        _apply_overrides(config, args.set)
        if not _run_startup_checks(config):
            return 2

        # M6: a cloud compute_target DISPATCHES to a Cloud Run Job instead of running in-
process.
        # DEFAULT-OFF: get_compute_backend returns None for ""/local_gpu/cpu_mock ? fall
through to the
        # unchanged inline path below. The dispatched container runs THIS cmd_infer inline
(the
        # dispatcher forces compute_target=local_gpu), so it never recursively re-
dispatches.
        compute = get_compute_backend(config)
        if compute is not None:
            return _dispatch_remote(compute, args)

        storage_cfg = config.storage.model_dump()
```

```
161
162         scratch = args.scratch or tempfile.mkdtemp(prefix="rally-worker-")
163         os.makedirs(scratch, exist_ok=True)
164         config.output.output_dir = scratch
165
166         # 1. PULL ? local:// / bare path = identity passthrough; s3:// / gcs:// = download
to scratch.
167         local_video = storage.fetch(args.video_uri, os.path.join(scratch, "in.mp4"),
config=storage_cfg)
168         print(f"[worker] pulled {args.video_uri} -> {local_video}")
169
170         # 2. IDENTITY ? whole-file MD5 (the labels?video join key; pulled bytes must be
byte-identical).
171         video_id = compute_video_id(local_video)
172
173         # 3. VALIDATE (same registry as the main CLI).
174         val_results, val_context = validator_registry.run_all_with_context(local_video,
config)
175         failures = [r for r in val_results if not r.passed]
176         db = Database(os.path.join(scratch, config.output.db_name))
177         db.add_video(video_id, local_video, "failed" if failures else "processed",
178                      [r.model_dump() for r in val_results])
179
180         segments: List[dict] = []
181         segmenter_actual = None
182         segmentation_ran = False
183         status = "failed"
184         try:
185             if failures:
186                 print("[worker] validation FAILED: "
187                       + "; ".join(f"{r.validator_name}: {r.message}" for r in failures))
188                 return 2
189             seg_name = args.segmenter or config.indexing.default_segmenter
190             segmenter_cls = segmenter_registry.get(seg_name)
191             if not segmenter_cls:
192                 print(f"[worker] unknown segmenter: {seg_name!r} "
193                       f"(have {list(segmenter_registry.list_available())})")
194                 return 2
195             segmenter_actual = seg_name
196             result = segmenter_cls(db, config).process_video(video_id, local_video,
max_frames=args.max_frames)
197             segmentation_ran = True
198             if isinstance(result, Failure):
199                 print(f"[worker] segmenter FAILED: {result.message}")
200                 return 3
201             segments = result.value if hasattr(result, "value") else result
202             status = "success"
203             # Loud, never-silent diagnostic: a 0-segment success is almost always a
misconfig, not an
204             # empty match ? the cold-start failure mode (the substrate detector yielded no
usable
205             # trajectories). Explain it so a run is analysable from the log alone (re-run -v
for more).
206             if not segments:
207                 detector_impl = (os.environ.get("RALLY_DETECTOR_IMPL")
208                                  or getattr(config.indexing, "detector_impl", None) or
"auto")
209                 logger.warning(
210                     "infer produced 0 segments for video_id=%s (segmenter=%s,
detector_impl=%s). "
211                     "The detector substrate yielded no usable trajectories/windows ? check
the detector "
212                     "logs above; common causes: native runner UNAVAILABLE
(weights/repo/WASB_PYTHON env), "
213                     "the WASB env produced no detections, or the input resolution differs
```

```
from the tuned "
                    "proxy resolution. Re-run with -v for the native command + frame
counts.",
                    video_id, segmenter_actual, detector_impl,
                )
        finally:
            # Best-effort per-video observability report (never raises) ? parity with the
main CLI.
            emit_indexer_report(
                db=db, video_id=video_id, video_path=local_video, config=config,
                val_results=val_results, val_context=val_context,
                segmenter_requested=args.segmenter, segmenter_actual=segmenter_actual,
                was_reingest=False, segmentation_ran=segmentation_ran,
            )

        # 5. SERIALIZE + 6. PUSH (outputs/<video_id>/?). idempotent on video_id.
        out_local = os.path.join(scratch, "outputs", video_id)
        os.makedirs(out_local, exist_ok=True)
        _write_intervals_csv(segments, os.path.join(out_local, "intervals.csv"))
        _write_report_json(video_id, segments, status, os.path.join(out_local,
"report.json"))

        out_prefix = args.out_uri.rstrip("/")
        pushed = []
        for fn in sorted(os.listdir(out_local)):
            uri = f"{out_prefix}/outputs/{video_id}/{fn}"
            storage.put(os.path.join(out_local, fn), uri, config=storage_cfg)
            pushed.append(uri)

        print(f"[worker] infer done: {len(segments)} segment(s); pushed {len(pushed)}
artifact(s):")
        for u in pushed:
            print("   ", u)

        # M6: a remote dispatcher passes --done-uri; write a completion marker so its
bucket-poll
        # terminates (carries video_id ? the dispatcher's output path + the worker rc).
DEFAULT-OFF:
        # no --done-uri ? no marker (unchanged local run). Written on SUCCESS only ? a
failed container
        # job currently lets the dispatcher's bounded poll time out (a failure-marker is a
follow-up).
        if getattr(args, "done_uri", None):
            _write_done_marker(args.done_uri, video_id, 0, len(segments), status,
storage_cfg, scratch)
        return 0


    #: Video extensions accepted under <corpus>/videos/ (mirrors the labels/ '.csv' filter
so a
    #: per-video sidecar ? .srt/.json/thumbnail ? can't shadow the real video on a stem
collision).
    _VIDEO_EXTS = (".mp4", ".mov", ".mkv", ".avi", ".webm", ".m4v")


    def _probe_video_fps_width(path: str):
        """Robust ffprobe (fps, frame_width) for a real video. Returns None on ANY failure.

        The detector trajectory is frame-indexed; the harness maps frame->time via fps, so a
WRONG
        fps silently mis-aligns every rally label (e.g. a 60fps clip read as 30 doubles
every time).
        cv2's ``_probe_fps_width`` silently defaults to (30, 1280) on a read miss ? fine for
the
        inference report (it flags the fallback) but corrupting for REAL training data. So
```

```
here we
264         probe with ffprobe (same tool as ``get_video_duration``) and return None so the
caller SKIPS
265         rather than guesses.
266         """
267         import json
268         import subprocess
269         try:
270             out = subprocess.check_output(
271                 ["ffprobe", "-v", "error", "-select_streams", "v:0",
272                  "-show_entries", "stream=r_frame_rate,width", "-of", "json", path],
273                 stderr=subprocess.DEVNULL).decode()
274             st = (json.loads(out).get("streams") or [{}])[0]
275             num, _, den = str(st.get("r_frame_rate", "0/1")).partition("/")
276             fps = float(num) / float(den or "1")
277             width = float(st.get("width") or 0)
278             if fps > 0 and width > 0:
279                 return fps, width
280         except (subprocess.SubprocessError, ValueError, KeyError, OSError,
json.JSONDecodeError):
281             pass
282         return None
283
284
285     def _resolve_train_detector(args: argparse.Namespace, config: Any):
286         """Build the trajectory DetectorRunner for `--trajectory-source detector`, honouring
287         detector_impl (RALLY_DETECTOR_IMPL / indexing.detector_impl: auto=WSL, native=GPU
box,
288         stub=CPU mock). Returns the runner, or prints an error + returns None.
289
290         Reuses the segmenter's `_resolve_runner` seam so the worker and the live CLI build
the
291         detector identically (one source of truth). A non-trajectory segmenter (e.g. yolo)
has
292         no shuttle detector and is rejected.
293         """
294         from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
295
296         seg_cls = segmenter_registry.get(args.segmenter)
297         if not seg_cls:
298             print(f"[worker] unknown segmenter: {args.segmenter!r} "
299                   f"(have {list(segmenter_registry.list_available())})")
300             return None
301         seg = seg_cls(None, config)
302         if not isinstance(seg, TrajectoryHybridSegmenter):
303             print(f"[worker] --segmenter {args.segmenter!r} has no shuttle detector; "
304                   f"use a trajectory hybrid (wasb_hybrid / tracknet_hybrid /
fusion_hybrid).")
305             return None
306         try:
307             runner, _ = seg._resolve_runner(config.get("indexing", {}))
308         except NotImplementedError as e:
309             # e.g. detector_impl='native' for a family without a native runner (tracknet, or
310             # fusion with a tracknet substrate). Fail cleanly (rc!=0), not with a raw
traceback.
311             print(f"[worker] detector not available: {e}")
312             return None
313         ok, msg = runner.healthcheck()
314         if not ok:
315             print(f"[worker] detector environment not ready: {msg}")
316             return None
317         print(f"[worker] detector ready ({args.segmenter}): {msg}")
318         return runner
319
320
```

```
    def cmd_train(args: argparse.Namespace) -> int:
        """Gen-0 train-from-bucket: pull labels (+ videos) -> trajectories -> manifest ->
shell out
        to training.gen0.harness (backend must NOT import training) -> push the artifact
bundle.

        Two trajectory sources (the train pipeline + harness are identical either way):
          * ``synth`` (default) ? the CPU "mock GPU": deterministic, label-consistent
synthetic
            shuttle paths (no GPU/WSL), so the whole pipeline is validated on a CPU box /
CI.
          * ``detector`` ? REAL shuttle trajectories: pull each video and run the resolved
            DetectorRunner (native WASB on a GPU box, or stub for a GPU-free wiring test).
This
            is the M3.5 "first real training" path; only the trajectory source flips.
        """
        import subprocess

        config = load_config(args.config) if args.config else load_config()
        _apply_overrides(config, args.set)
        if not _run_startup_checks(config):
            return 2
        storage_cfg = config.storage.model_dump()

        scratch = args.scratch or tempfile.mkdtemp(prefix="rally-train-")
        labels_dir = os.path.join(scratch, "labels")
        traj_dir = os.path.join(scratch, "trajectories")
        videos_dir = os.path.join(scratch, "videos")
        os.makedirs(labels_dir, exist_ok=True)
        os.makedirs(traj_dir, exist_ok=True)

        corpus = args.corpus_uri.rstrip("/")
        label_uris = sorted(u for u in storage.list_uris(f"{corpus}/labels/",
config=storage_cfg)
                            if u.lower().endswith(".csv"))
        if len(label_uris) < 2:
            print(f"[worker] need >=2 labeled videos for leave-one-video-out; found
{len(label_uris)} "
                  f"under {corpus}/labels/")
            return 2

        # Real-detector source: resolve the runner once + index the bucket's videos/ by
stem.
        runner = None
        video_by_stem: dict = {}
        if args.trajectory_source == "detector":
            os.makedirs(videos_dir, exist_ok=True)
            runner = _resolve_train_detector(args, config)
            if runner is None:
                return 2
            for vuri in storage.list_uris(f"{corpus}/videos/", config=storage_cfg):
                vname = storage.parse_uri(vuri).name
                if not vname.lower().endswith(_VIDEO_EXTS):
                    continue  # skip sidecars (.srt/.json/thumbnails) so they can't shadow
the video
                video_by_stem[os.path.splitext(vname)[0]] = vuri

        print(f"[worker] trajectory source: {args.trajectory_source}")
        specs: List[dict] = []
        for uri in label_uris:
            fname = storage.parse_uri(uri).name                    # e.g.
GX020094_proxy.rallies.csv
            name = fname.replace(".rallies.csv", "").replace(".csv", "")
            local_label = storage.fetch(uri, os.path.join(labels_dir, fname),
config=storage_cfg)
```

```
            if args.trajectory_source == "detector":
                vuri = video_by_stem.get(name)
                if not vuri:
                    print(f"[worker] no video for {name!r} under {corpus}/videos/ ?
skipping.")
                    continue
                local_video = storage.fetch(vuri, os.path.join(videos_dir,
storage.parse_uri(vuri).name),
                                            config=storage_cfg)
                video_id = compute_video_id(local_video)
                # Real fps/width drive the trajectory frame->time mapping; PROBE (don't
guess) ? a
                # wrong fps silently mis-aligns every label, so skip the video if ffprobe
can't read it.
                meta = _probe_video_fps_width(local_video)
                if meta is None:
                    print(f"[worker] could not probe fps/width for {name!r} (ffprobe) ?
skipping (won't guess).")
                    continue
                fps, frame_width = meta
                traj = runner.run_predict(local_video, traj_dir, video_id=video_id)
                if not traj:
                    print(f"[worker] detector produced no trajectory for {name!r} ?
skipping.")
                    continue
                specs.append({"name": name, "trajectory": traj, "labels": local_label,
                              "fps": fps, "frame_width": frame_width})
            else:
                from backend.pipeline.detectors.stub_runner import
synth_trajectory_from_labels
                traj = os.path.join(traj_dir, f"{name}_traj.csv")
                synth_trajectory_from_labels(local_label, traj, fps=args.fps,
frame_width=args.frame_width)
                specs.append({"name": name, "trajectory": traj, "labels": local_label,
                              "fps": args.fps, "frame_width": args.frame_width})

        if len(specs) < 2:
            print(f"[worker] need >=2 videos with trajectories for leave-one-video-out; got
{len(specs)}.")
            return 2
        manifest_path = os.path.join(scratch, "manifest.json")
        with open(manifest_path, "w", encoding="utf-8") as f:
            json.dump(specs, f, indent=2)
        src_label = "real detector" if args.trajectory_source == "detector" else "CPU-mock
stub"
        print(f"[worker] built manifest: {len(specs)} videos ({src_label} trajectories)")

        out_dir = os.path.join(scratch, "artifacts")
        cmd = [sys.executable, "-m", "training.gen0.harness",
               "--manifest", manifest_path, "--out", out_dir, "--version", args.version]
        if args.floor:
            floor_local = (storage.fetch(args.floor, os.path.join(scratch, "floor.json"),
config=storage_cfg)
                           if "://" in args.floor else args.floor)
            cmd += ["--floor", floor_local]
        print("[worker] training:", " ".join(cmd))
        rc = subprocess.run(cmd).returncode
        if rc != 0:
            print(f"[worker] gen0 harness failed (rc={rc})")
            return rc

        # PUSH the versioned artifact bundle (weights.json + manifest.json) to the bucket.
        bundle = os.path.join(out_dir, args.version)
        out_prefix = args.out_uri.rstrip("/")
        pushed = []
```

```
        for fn in sorted(os.listdir(bundle)):
            uri = f"{out_prefix}/weights/{args.version}/{fn}"
            storage.put(os.path.join(bundle, fn), uri, config=storage_cfg)
            pushed.append(uri)
        print(f"[worker] train done: pushed {len(pushed)} artifact(s):")
        for u in pushed:
            print("   ", u)
        return 0


    def build_parser() -> argparse.ArgumentParser:
        p = argparse.ArgumentParser(prog="python -m backend.worker",
                                    description="Storage-aware worker: pull ? run pipeline ?
push.")
        p.add_argument("-v", "--verbose", action="store_true",
                       help="DEBUG logging (the native detector command, frame counts, per-
stage detail)")
        sub = p.add_subparsers(dest="cmd", required=True)

        pi = sub.add_parser("infer", help="run inference on one video URI and push outputs")
        pi.add_argument("--video-uri", required=True, help="local path / local:// / s3:// /
gcs://")
        pi.add_argument("--out-uri", required=True, help="output prefix
(outputs/<video_id>/? is appended)")
        pi.add_argument("--segmenter", default=None, help="default:
indexing.default_segmenter")
        pi.add_argument("--config", default=None, help="config.json path (default:
./config.json)")
        pi.add_argument("--scratch", default=None, help="local scratch dir (default: a temp
dir)")
        pi.add_argument("--max-frames", type=int, default=None)
        pi.add_argument("--done-uri", default=None,
                        help="M6: storage URI for a completion MARKER (video_id + rc)
written on a "
                             "successful run, so a remote (Cloud Run) dispatcher's bucket-
poll terminates. "
                             "Default-OFF: omit it for the normal local run.")
        pi.add_argument("--set", action="append", default=[], metavar="k=v",
                        help="config override, e.g. --set indexing.skip_ai_handoff=true")
        pi.set_defaults(func=cmd_infer)

        pt = sub.add_parser("train", help="Gen-0 train-from-bucket (labels [+videos] ->
trajectories -> harness)")
        pt.add_argument("--corpus-uri", required=True, help="prefix containing labels/ (+
videos/ for "
                        "--trajectory-source detector), e.g. s3://bucket or gcs://bucket")
        pt.add_argument("--out-uri", required=True, help="output prefix (weights/<version>/
is appended)")
        pt.add_argument("--version", required=True, help="semver for the artifact bundle")
        pt.add_argument("--trajectory-source", choices=["synth", "detector"],
default="synth",
                        help="synth = CPU-mock label-consistent trajectories (default, no
GPU); "
                             "detector = REAL shuttle trajectories via the resolved
DetectorRunner "
                             "(needs videos/ in the corpus; honours detector_impl).")
        pt.add_argument("--segmenter", default="wasb_hybrid",
                        help="trajectory hybrid whose detector is used for --trajectory-
source detector")
        pt.add_argument("--floor", default=None, help="floor baseline JSON (local path or
storage URI)")
        pt.add_argument("--config", default=None)
        pt.add_argument("--scratch", default=None)
        pt.add_argument("--fps", type=float, default=30.0, help="synth trajectory fps
(detector probes the video)")
```

```
476         pt.add_argument("--frame-width", type=float, default=1280.0,
477                         help="synth trajectory frame width (detector probes the video)")
478         pt.add_argument("--set", action="append", default=[], metavar="k=v")
479         pt.set_defaults(func=cmd_train)
480         return p
481
482
483     def _setup_logging(verbose: bool) -> None:
484         """Configure logging for the worker CLI so the pipeline's existing INFO/ERROR
diagnostics are
485         VISIBLE ? the worker previously ran with no logging config, so every
segmenter/detector log
486         (detector choice, 'Running native WASB inference', availability failures) was
silently dropped.
487         That is exactly why a 0-rally cold-start gave nothing to analyse. Configured here
(the CLI
488         entry), not at import, so importing the module never reconfigures a caller's
logging."""
489         # force=True: basicConfig is a no-op if the root already has handlers (e.g. a
library
490         # configured logging at import), which would leave the level at WARNING and re-
suppress the
491         # diagnostics. Forcing reconfiguration means the worker reliably controls its own
verbosity.
492         logging.basicConfig(
493             level=logging.DEBUG if verbose else logging.INFO,
494             format="%(asctime)s [%(levelname)s] %(name)s: %(message)s",
495             datefmt="%H:%M:%S",
496             force=True,
497         )
498
499
500     def main(argv=None) -> int:
501         args = build_parser().parse_args(argv)
502         _setup_logging(getattr(args, "verbose", False))
503         return args.func(args)
504
505
506     if __name__ == "__main__":
507         sys.exit(main())
508
```

9. assistant (2026-06-21T10:45:29.997Z)

Now let me read the execution_policy (poll_until / PolicyTimeout), the storage layer functions, the deploy files, and the test file.

10. user (2026-06-21T10:45:30.966Z)

```
1       """Per-job execution policy: bounded, observable waits for long/flaky ops
(EXECUTION_POLICY.md P1).
2
3       Turns an unbounded blocking call (the GX010125 Gemini hang) into a bounded one:
4       - ``run_bounded`` ? a HARD per-attempt timeout (``op_timeout_sec``, the hang-killer) +
retry-with-backoff
5         for FAILURES (an exception) *and* HANGS (no return in time). A hung attempt's worker
thread is a daemon,
6         so it is abandoned, not joined ? it never blocks the process.
7       - ``poll_until`` ? a bounded poll loop for an in-progress external op
(``poll_every_n_sec`` cadence,
8         ``max_wait_sec`` deadline).
9
10      **Testability:** ``sleep``/``monotonic`` are injected, so the whole module is unit-
tested with zero real
11      waiting (a fake clock advances instantly).
```

```
12
13      **Logging discipline** (so polling is VISIBLE but never NOISY): every poll/retry logs at
**DEBUG**; state
14      transitions (start, resolved, hung, gave-up) log at **INFO/WARNING**. INFO stays clean;
enable DEBUG on the
15      ``execution.policy`` logger to watch the poll trail.
16      """
17      from __future__ import annotations
18
19      import logging
20      import threading
21      import time as _time
22      from dataclasses import asdict, dataclass
23      from enum import Enum
24      from typing import Callable, Optional, TypeVar
25
26      T = TypeVar("T")
27
28      #: Dedicated logger so callers can dial polling visibility independently of the app's
root level.
29      LOG = logging.getLogger("execution.policy")
30
31
32      class WaitMode(str, Enum):
33          """How a job treats a long wait. ``str``-valued so it serialises straight to JSON on
the job row."""
34          WAIT_AND_RESUME = "wait_and_resume"   # default: bounded wait; over budget ->
reschedule/partial (P2/P3)
35          ABORT = "abort"                       # one attempt; on hang/fail, stop immediately
36          BLOCK = "block"                       # bounded in-process blocking wait (no
reschedule)
37
38
39      class PolicyTimeout(TimeoutError):
40          """An op hung past ``op_timeout_sec`` or never became ready within
``max_wait_sec``."""
41
42
43      @dataclass(frozen=True)
44      class ExecutionPolicy:
45          """Declarative resilience knobs, attachable per job/op (JSON-serialisable; see
EXECUTION_POLICY.md)."""
46          mode: WaitMode = WaitMode.WAIT_AND_RESUME
47          poll_every_n_sec: float = 10.0
48          op_timeout_sec: float = 120.0         # HARD per-attempt deadline ? the hang-killer
that was missing
49          max_wait_sec: float = 1800.0          # total wall budget for a poll loop
50          max_retries: int = 5                  # retries (=> max_retries+1 attempts) for
failures/hangs
51          backoff_base_sec: float = 3.0         # exponential: backoff_base * 2**attempt
52          on_timeout: str = "reschedule"        # reschedule | partial | fail  (consumed by
P2/P3)
53          resumable: bool = True
54
55          def to_dict(self) -> dict:
56              d = asdict(self)
57              d["mode"] = self.mode.value
58              return d
59
60          @classmethod
61          def from_dict(cls, d: dict) -> "ExecutionPolicy":
62              known = {f for f in cls.__dataclass_fields__}  # type: ignore[attr-defined]
63              kw = {k: v for k, v in d.items() if k in known}
64              if "mode" in kw and not isinstance(kw["mode"], WaitMode):
65                  kw["mode"] = WaitMode(kw["mode"])
```

```
            return cls(**kw)


    #: The sane default (also what a job gets if it declares no policy).
    DEFAULT_POLICY = ExecutionPolicy()


    def _backoff_sec(policy: ExecutionPolicy, attempt: int) -> float:
        """Deterministic exponential backoff (no jitter ? reproducible tests)."""
        return policy.backoff_base_sec * (2 ** attempt)


    def run_bounded(fn: Callable[[], T], policy: ExecutionPolicy = DEFAULT_POLICY, *,
                    label: str = "op", logger: Optional[logging.Logger] = None,
                    sleep: Callable[[float], None] = _time.sleep) -> T:
        """Run ``fn()`` under a hard per-attempt timeout, retrying FAILURES and HANGS with backoff.

        Returns ``fn``'s value on success. Raises ``PolicyTimeout`` if the final attempt hung, or re-raises the
        last exception if the final attempt failed. ``mode=ABORT`` => a single attempt (no retries).
        """
        log = logger or LOG
        attempts = 1 if policy.mode is WaitMode.ABORT else policy.max_retries + 1
        last_exc: Optional[BaseException] = None
        for attempt in range(attempts):
            box: dict = {}
            done = threading.Event()

            # box/done bound as defaults (not closed over) ? the thread is started + joined within this
            # same iteration, but explicit binding keeps each attempt isolated and silences B023.
            def _runner(_box: dict = box, _done: threading.Event = done) -> None:
                try:
                    _box["val"] = fn()
                except BaseException as e:        # relay ANY error (incl. timeouts) to the caller thread
                    _box["exc"] = e
                finally:
                    _done.set()

            threading.Thread(target=_runner, name=f"bounded:{label}", daemon=True).start()
            if done.wait(timeout=policy.op_timeout_sec):
                if "exc" in box:
                    last_exc = box["exc"]
                    log.info("%s: attempt %d/%d failed: %s", label, attempt + 1, attempts, last_exc)
                else:
                    if attempt:
                        log.info("%s: succeeded on attempt %d/%d", label, attempt + 1, attempts)
                    return box["val"]              # type: ignore[return-value]
            else:
                last_exc = PolicyTimeout(f"{label} hung > op_timeout_sec={policy.op_timeout_sec}s")
                log.warning("%s: attempt %d/%d HUNG (>%.0fs) ? abandoning", label, attempt + 1, attempts,
                            policy.op_timeout_sec)
            if attempt < attempts - 1:
                wait = _backoff_sec(policy, attempt)
                log.debug("%s: backing off %.1fs before retry", label, wait)
                sleep(wait)
        assert last_exc is not None
```

```
121         log.warning("%s: gave up after %d attempt(s): %s", label, attempts, last_exc)
122         raise last_exc
123
124
125     def poll_until(check: Callable[[], Optional[T]], policy: ExecutionPolicy =
DEFAULT_POLICY, *,
126                    label: str = "op", logger: Optional[logging.Logger] = None,
127                    sleep: Callable[[float], None] = _time.sleep,
128                    monotonic: Callable[[], float] = _time.monotonic) -> T:
129         """Poll ``check()`` every ``poll_every_n_sec`` until it returns a non-``None``
value, or raise
130         ``PolicyTimeout`` after ``max_wait_sec``. ``check`` returns ``None`` while the op is
still in progress.
131
132         Polls log at DEBUG (no INFO spam); the wait start + resolution + give-up log at
INFO/WARNING.
133         """
134         log = logger or LOG
135         start = monotonic()
136         polls = 0
137         log.info("%s: waiting (poll_every=%.0fs, max_wait=%.0fs)",
138                  label, policy.poll_every_n_sec, policy.max_wait_sec)
139         while True:
140             result = check()
141             if result is not None:
142                 log.info("%s: ready after %d poll(s) / %.0fs", label, polls, monotonic() -
start)
143                 return result
144             elapsed = monotonic() - start
145             if elapsed >= policy.max_wait_sec:
146                 log.warning("%s: still not ready after %.0fs (max_wait) ? giving up", label,
elapsed)
147                 raise PolicyTimeout(f"{label} not ready within
max_wait_sec={policy.max_wait_sec}s")
148             polls += 1
149             log.debug("%s: poll %d, %.0fs elapsed, not ready", label, polls, elapsed)
150             sleep(policy.poll_every_n_sec)
151
```

11. user (2026-06-21T10:45:31.441Z)

```
1       """ComputeBackend seam ? Cloud Run dispatch shim (COMPUTE_DECOUPLED_SERVING M6).
2
3       All GPU-free + cloud-free: the Cloud Run trigger is a FAKE ``CloudRunJobClient`` that
simulates the
4       worker (writes the done-marker + report to a local "bucket" = a tmp dir), so the
dispatch?bucket-poll
5       ?read flow is proven for $0. The cardinal invariant: **default-OFF** ? only
``compute_target=cloud_run``
6       selects a backend; everything else returns ``None`` (= run the worker's existing in-
process path).
7       """
8       import json
9       import os
10
11      from backend.compute import ComputeJobSpec, get_compute_backend
12      from backend.compute.cloud_run import CloudRunCompute, CloudRunJobClient
13      from backend.config import AppConfig
14      from backend.infrastructure.execution_policy import ExecutionPolicy
15
16      # A fast, no-wait policy (the fake client writes the marker synchronously, so the first
poll hits).
17      _FAST = ExecutionPolicy(poll_every_n_sec=0.0, op_timeout_sec=5.0, max_wait_sec=5.0)
18
19
```

```
    class _FakeRunClient(CloudRunJobClient):
        """Simulates a Cloud Run Job: on dispatch, write the report + done-marker into the local bucket
        exactly where the real worker would, so CloudRunCompute's bucket-poll resolves immediately."""

        def __init__(self, *, video_id="vidCloud", segment_count=2, returncode=0):
            self.calls: list[dict] = []
            self.video_id, self.segment_count, self.returncode = video_id, segment_count, returncode

        def run_job(self, job_name, argv, *, region, project=None):
            self.calls.append({"job": job_name, "argv": list(argv), "region": region, "project": project})
            out_uri = argv[argv.index("--out-uri") + 1].rstrip("/")
            done_uri = argv[argv.index("--done-uri") + 1]
            outputs = os.path.join(out_uri, "outputs", self.video_id)
            os.makedirs(outputs, exist_ok=True)
            with open(os.path.join(outputs, "report.json"), "w", encoding="utf-8") as f:
                json.dump({"video_id": self.video_id, "segment_count": self.segment_count,
                           "status": "success"}, f)
            os.makedirs(os.path.dirname(done_uri), exist_ok=True)
            with open(done_uri, "w", encoding="utf-8") as f:
                json.dump({"video_id": self.video_id, "returncode": self.returncode,
                           "segment_count": self.segment_count, "status": "success"}, f)
            return "exec-123"


    # ---------------------------------------------------------------------- factory (default-OFF)

    def _cfg(compute_target="", storage_backend="local"):
        return AppConfig.model_validate({
            "deployment": {"compute_target": compute_target},
            "storage": {"backend": storage_backend},
        })


    def test_factory_is_none_for_local_targets():
        """DEFAULT-OFF: no/local/cpu_mock compute target ? None (run the in-process path)."""
        assert get_compute_backend(_cfg("")) is None
        assert get_compute_backend(_cfg("local_gpu")) is None
        assert get_compute_backend(_cfg("cpu_mock")) is None


    def test_factory_returns_cloud_run_for_cloud_target():
        backend = get_compute_backend(_cfg("cloud_run", "gcs"), run_client=_FakeRunClient())
        assert isinstance(backend, CloudRunCompute)


    # ---------------------------------------------------------------- CloudRunCompute dispatch + poll

    def test_cloud_run_dispatch_polls_and_returns_result(tmp_path):
        client = _FakeRunClient(video_id="abc123", segment_count=3)
        backend = CloudRunCompute(run_client=client, job_name="rally-worker", region="asia-south1",
                                  project="proj", storage_config={}, policy=_FAST, token="tok")
        out = str(tmp_path / "bucket")
        result = backend.run_infer(ComputeJobSpec(
            video_uri="gs://b/videos/clip.mp4", out_uri=out, segmenter="wasb_hybrid"))

        assert result.returncode == 0
```

```
        assert result.video_id == "abc123"
        assert result.outputs_uri == f"{out}/outputs/abc123/"
        assert result.report.get("segment_count") == 3
        # exactly one Cloud Run Job execution was triggered, in the right region/project
        assert len(client.calls) == 1
        assert client.calls[0]["job"] == "rally-worker" and client.calls[0]["region"] ==
"asia-south1"


    def test_dispatch_argv_forces_inline_and_carries_overrides(tmp_path):
        """The dispatched container must run INLINE (never re-dispatch) ?
compute_target=local_gpu +
        native detector appended AFTER the user overrides (so they win); --done-uri always
present."""
        client = _FakeRunClient()
        backend = CloudRunCompute(run_client=client, job_name="j", region="r", policy=_FAST,
token="tok")
        backend.run_infer(ComputeJobSpec(
            video_uri="gs://b/v.mp4", out_uri=str(tmp_path / "bk"), segmenter="wasb_hybrid",
            config_overrides=("sport=tennis",)))

        argv = client.calls[0]["argv"]
        assert argv[0] == "infer"
        assert "--done-uri" in argv and "--video-uri" in argv and "--out-uri" in argv
        assert argv[argv.index("--segmenter") + 1] == "wasb_hybrid"
        # the container-inline overrides come last (win) and the user override is carried
through
        joined = " ".join(argv)
        assert "deployment.compute_target=local_gpu" in joined
        assert "indexing.detector_impl=native" in joined
        assert "sport=tennis" in joined
        # user override precedes the inline-forcing overrides in --set order
        assert joined.index("sport=tennis") <
joined.index("deployment.compute_target=local_gpu")


    def test_unique_token_per_call_when_not_injected(tmp_path):
        """No injected token ? a fresh uuid per dispatch, so concurrent jobs don't collide
on the marker."""
        client = _FakeRunClient()
        backend = CloudRunCompute(run_client=client, job_name="j", region="r", policy=_FAST)
        backend.run_infer(ComputeJobSpec(video_uri="gs://b/v.mp4", out_uri=str(tmp_path /
"bk")))
        backend.run_infer(ComputeJobSpec(video_uri="gs://b/v.mp4", out_uri=str(tmp_path /
"bk")))
        done1 = client.calls[0]["argv"][client.calls[0]["argv"].index("--done-uri") + 1]
        done2 = client.calls[1]["argv"][client.calls[1]["argv"].index("--done-uri") + 1]
        assert done1 != done2


    # ------------------------------------------------- worker wiring: dispatch branch +
done-marker

    def test_cmd_infer_dispatches_when_cloud_target(tmp_path, monkeypatch):
        """cmd_infer with compute_target=cloud_run hands off to the ComputeBackend (no in-
process run)."""
        from backend.compute.base import ComputeResult
        from backend.worker import __main__ as wmain

        seen = {}

        class _FakeBackend:
            def run_infer(self, spec):
                seen["spec"] = spec
                return ComputeResult(returncode=0, video_id="vidX",
```

```
outputs_uri="gs://b/outputs/vidX/")
130
131         monkeypatch.setattr(wmain, "get_compute_backend", lambda config: _FakeBackend())
132         cfg = tmp_path / "c.json"
133         cfg.write_text('{"deployment": {"profile": "cloud-serving"}, "storage": {"backend":
"gcs"}}')
134         rc = wmain.cmd_infer(wmain.build_parser().parse_args([
135             "infer", "--video-uri", "gs://b/videos/clip.mp4", "--out-uri", "gs://b",
136             "--config", str(cfg), "--segmenter", "wasb_hybrid"]))
137         assert rc == 0
138         assert seen["spec"].video_uri == "gs://b/videos/clip.mp4"
139         assert seen["spec"].segmenter == "wasb_hybrid"
140
141
142     def test_worker_writes_done_marker_on_success(tmp_path, monkeypatch):
143         """The container side: cmd_infer with --done-uri writes a completion marker
(video_id + rc) so a
144         remote dispatcher's bucket-poll terminates. Driven by the deterministic stub + skip-
AI."""
145         from backend.worker import __main__ as wmain
146
147         class _OK:
148             passed = True
149             validator_name = "fake"
150             message = "ok"
151
152             def model_dump(self):
153                 return {"validator_name": "fake", "passed": True, "message": "ok"}
154
155         monkeypatch.setattr(wmain.validator_registry, "run_all_with_context",
156                             lambda path, cfg: ([_OK()], {}))
157         monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
158
monkeypatch.setattr("backend.pipeline.segmenters.trajectory_hybrid.get_video_duration",
159                             lambda p: 30.0)
160
161         cfg = tmp_path / "c.json"
162         cfg.write_text(json.dumps({"indexing": {"detector_impl": "stub", "skip_ai_handoff":
True},
163                                    "output": {"output_dir": str(tmp_path / "o")}}))
164         video = tmp_path / "in.mp4"
165         video.write_bytes(b"FAKEVIDEO")
166         out = tmp_path / "bucket"
167         done = str(out / "_dispatch" / "tok.done")
168
169         rc = wmain.cmd_infer(wmain.build_parser().parse_args([
170             "infer", "--video-uri", str(video), "--out-uri", str(out), "--config", str(cfg),
171             "--scratch", str(tmp_path / "s"), "--segmenter", "wasb_hybrid", "--done-uri",
done]))
172         assert rc == 0
173         marker = json.loads(open(done, encoding="utf-8").read())
174         assert marker["returncode"] == 0 and marker["status"] == "success"
175         assert marker["video_id"] and marker["segment_count"] >= 1
176
177
178     def test_worker_no_done_uri_is_unchanged(tmp_path, monkeypatch):
179         """DEFAULT-OFF: without --done-uri the inline run writes NO marker (today's
behaviour)."""
180         from backend.worker import __main__ as wmain
181
182         class _OK:
183             passed = True
184             validator_name = "fake"
185             message = "ok"
186
```

```
187             def model_dump(self):
188                 return {"validator_name": "fake", "passed": True, "message": "ok"}
189
190         monkeypatch.setattr(wmain.validator_registry, "run_all_with_context",
191                             lambda path, cfg: ([_OK()], {}))
192         monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
193
monkeypatch.setattr("backend.pipeline.segmenters.trajectory_hybrid.get_video_duration",
194                             lambda p: 30.0)
195         cfg = tmp_path / "c.json"
196         cfg.write_text(json.dumps({"indexing": {"detector_impl": "stub", "skip_ai_handoff":
True},
197                                    "output": {"output_dir": str(tmp_path / "o")}}))
198         video = tmp_path / "in.mp4"
199         video.write_bytes(b"FAKEVIDEO")
200         out = tmp_path / "bucket"
201         rc = wmain.cmd_infer(wmain.build_parser().parse_args([
202             "infer", "--video-uri", str(video), "--out-uri", str(out), "--config", str(cfg),
203             "--scratch", str(tmp_path / "s"), "--segmenter", "wasb_hybrid"]))
204         assert rc == 0
205         assert not (out / "_dispatch").exists()  # no marker dir when --done-uri is omitted
206
207
208     # ------------------------------------------------------------ Dockerfile structural guard
(no build)
209
210     def test_dockerfile_has_required_directives():
211         """CI can't build a CUDA image, so guard the Dockerfile's load-bearing directives
structurally:
212         the GPU base, ffmpeg + DejaVu fonts (the watermark), the WASB conda env, and the
worker entrypoint."""
213         from pathlib import Path
214
215         text = Path("deploy/cloudrun/Dockerfile").read_text(encoding="utf-8")
216         assert "FROM nvidia/cuda" in text                       # GPU base (CUDA)
217         assert "ffmpeg" in text                                  # stitch/decode
218         assert "fonts-dejavu" in text                            # the watermark filter font
219         assert "torch==1.11.0+cu113" in text                    # the verified WASB stack
220         assert "WASB-SBDT" in text                               # the shuttle detector repo
221         assert 'ENTRYPOINT ["python3", "-m", "backend.worker"]' in text
222         assert "WASB_PYTHON=/opt/conda/envs/wasb/bin/python" in text
223
```

12. user (2026-06-21T10:45:32.386Z)

```
1       # Cloud Run GPU **Job** image for the rally worker (COMPUTE_DECOUPLED_SERVING M6/M7,
ADR-012/D16).
2       #
3       # Runs `python -m backend.worker infer ?` INLINE on an L4 (detect + window + stitch co-
located so
4       # CPU/wall + egress are metered into one job ? D4). Built + pushed + verified by the
owner on the
5       # M7 real run (the $100/?10k budget, D17); CI does NOT build this (a CUDA image is too
heavy) ? the
6       # dispatch shim is proven with a fake client (tests/test_compute_backend.py).
7       #
8       # Two python stacks, deliberately separate (a framework conflict otherwise ? same split
as the
9       # native GPU box, deploy/digitalocean/gpu_setup.sh):
10      #   * the MAIN app env (this repo + ffmpeg) drives the pipeline + stitching;
11      #   * a conda `wasb` env (py3.8 + torch 1.11.0+cu113) runs the WASB shuttle detector as
a subprocess
12      #     (the native runner shells out to $WASB_PYTHON), so the app's torch never co-
installs with it.
13      #
```

```
14      # ? WASB-C3: the pretrained badminton weights are academic-data-trained ? provenance NOT
cleared for
15      #   commercial sharing. They are NOT baked into this image; stage them at runtime (a
bucket fetch or a
16      #   mount) and point $WASB_WEIGHTS at them. Resolve WASB-C3 before any public, non-owner
serving.
17
18      FROM nvidia/cuda:12.4.1-runtime-ubuntu22.04
19
20      ENV DEBIAN_FRONTEND=noninteractive \
21          PYTHONUNBUFFERED=1 \
22          PIP_NO_CACHE_DIR=1 \
23          CONDA_DIR=/opt/conda \
24          WASB_REPO_DIR=/opt/models/WASB-SBDT
25
26      # ffmpeg (stitch/decode) + DejaVu fonts (the watermark filter) + git/curl for the WASB
env build.
27      RUN apt-get update && apt-get install -y --no-install-recommends \
28              ffmpeg fonts-dejavu-core git curl ca-certificates build-essential \
29              python3 python3-pip python3-venv \
30          && rm -rf /var/lib/apt/lists/*
31
32      # --- WASB detector env (mirrors deploy/digitalocean/gpu_setup.sh ? the verified
torch-1.11 stack) ---
33      RUN curl -sSL https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh -o
/tmp/mc.sh \
34          && bash /tmp/mc.sh -b -p "$CONDA_DIR" && rm -f /tmp/mc.sh \
35          && "$CONDA_DIR/bin/conda" tos accept --override-channels --channel
https://repo.anaconda.com/pkgs/main || true \
36          && "$CONDA_DIR/bin/conda" tos accept --override-channels --channel
https://repo.anaconda.com/pkgs/r || true \
37          && "$CONDA_DIR/bin/conda" create -y -q -n wasb python=3.8 \
38          && git clone --depth 1 https://github.com/nttcom/WASB-SBDT.git "$WASB_REPO_DIR" \
39          && "$CONDA_DIR/envs/wasb/bin/python" -m pip install -q \
40              torch==1.11.0+cu113 torchvision==0.12.0+cu113 --extra-index-url
https://download.pytorch.org/whl/cu113 \
41          && ( [ -f "$WASB_REPO_DIR/requirements.txt" ] && "$CONDA_DIR/envs/wasb/bin/python"
-m pip install -q -r "$WASB_REPO_DIR/requirements.txt" || true ) \
42          && "$CONDA_DIR/envs/wasb/bin/python" -m pip install -q \
43              hydra-core omegaconf pandas opencv-python-headless pillow scipy tqdm pyyaml
matplotlib "numpy<1.24"
44
45      # --- Main application env ---
46      WORKDIR /app
47      COPY pyproject.toml README.md ./
48      COPY backend ./backend
49      COPY training ./training
50      RUN python3 -m pip install --upgrade pip && python3 -m pip install .
51
52      # The native runner finds the WASB env + repo by env (no `conda activate` needed);
weights come in at
53      # runtime (WASB-C3). detector_impl=native = the Linux-native WASB runner; cuda stays the
default.
54      ENV WASB_PYTHON=/opt/conda/envs/wasb/bin/python \
55          WASB_REPO_DIR=/opt/models/WASB-SBDT \
56          WASB_WEIGHTS=/opt/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar \
57          RALLY_DETECTOR_IMPL=native
58
59      # The dispatcher (CloudRunCompute) supplies the per-execution args:
60      #   infer --video-uri gs://? --out-uri gs://? --done-uri gs://? --segmenter wasb_hybrid
--set ?
61      ENTRYPOINT ["python3", "-m", "backend.worker"]
62
```

13. user (2026-06-21T10:45:32.497Z)

```
# Cloud Run GPU serving ? build / deploy / run (COMPUTE_DECOUPLED_SERVING M6/M7)

The cloud-serving compute path: a control plane (`CloudRunCompute`,
`backend/compute/cloud_run.py`)
dispatches a Cloud Run **Job** execution running this image's worker on an **L4**, then
**bucket-polls**
a done-marker for completion (D16). Scale-to-zero (D7) ? no warm pool.

> **Status:** the dispatch shim + image are **landed + mock-tested** (M6). The **real
build + push +
> run on an L4** is the **M7** owner step, within the **$100/?10k** budget (D17) ?
confirm cmd + hourly
> cost first, **delete every resource after** ([[gcp-vm-always-delete]]). CI does
**not** build this image.

## 0. Prerequisites (owner)
- A GCP project with billing on + the Cloud Run, Artifact Registry, and Cloud Build APIs
enabled.
- GPU quota for L4 (`GPUS_ALL_REGIONS` ? 1) in the chosen region (asia-south1, else
Singapore ? see
  `deploy/gcp/README.md`; L4 is often stocked-out, fall back per that runbook).
- The WASB weights staged in the bucket (WASB-C3 unresolved ? owner-gated):
`gs://<bucket>/models/wasb_badminton_best.pth.tar`.

## 1. Build + push the image (Artifact Registry)
```bash
PROJECT=<gcp-project>; REGION=asia-south1; REPO=rally; IMG=rally-worker
gcloud artifacts repositories create $REPO --repository-format=docker --location=$REGION
2>/dev/null || true
gcloud builds submit --tag $REGION-docker.pkg.dev/$PROJECT/$REPO/$IMG:latest -f
deploy/cloudrun/Dockerfile .
```
(A CUDA + conda image is large; the first build is slow. Budget-watch per D17.)

2. Create the Cloud Run **Job** (GPU, scale-to-zero)
```bash
gcloud run jobs create rally-worker \
  --image $REGION-docker.pkg.dev/$PROJECT/$REPO/$IMG:latest \
  --region $REGION --gpu 1 --gpu-type nvidia-l4 --cpu 4 --memory 16Gi \
  --max-retries 0 --task-timeout 3600 \
  --set-env-vars WASB_WEIGHTS=/staged/wasb_badminton_best.pth.tar
# (stage the weights into the container at start, or fetch from gs:// in an init step ?
WASB-C3.)
```

3. Wire the dispatcher (control plane)
The control plane runs with the **cloud-serving** profile (`DEPLOYMENT_PROFILE=cloud-
serving` ?
`compute_target=cloud_run`). `get_compute_backend(config)` then builds `CloudRunCompute`
from env:
```bash
export DEPLOYMENT_PROFILE=cloud-serving
export CLOUD_RUN_JOB=rally-worker  CLOUD_RUN_REGION=$REGION
GOOGLE_CLOUD_PROJECT=$PROJECT
# A `worker infer` (or the API job path) with a gs:// video now DISPATCHES to the Job
instead of
# running in-process; the container runs the SAME worker INLINE (compute_target forced
local_gpu).
python -m backend.worker infer --video-uri gs://<bucket>/videos/clip.mp4 --out-uri
gs://<bucket> --segmenter wasb_hybrid
```

4. M7 acceptance (the owner runlog)
Capture a `DEPLOY_REPORT_cloud-serving_<date>.md` under
`docs/COMPUTE_DECOUPLED_SERVING/runlogs/` (mirrors the truly-local template): the exact
```

```
commands,
49      rally counts, `gpu_active_seconds`/`wall_seconds`/`bytes_out`, a reel smoke-test
(`ffmpeg -f null -`
50      exit 0 + a 5-frame contact sheet), gotchas, and the **$0 teardown audit**.
51
52      ## Gotchas
53      - The dispatched container **must run inline** ? `CloudRunCompute` forces
`compute_target=local_gpu`
54        + `indexing.detector_impl=native` in the per-execution args so the Job never re-
dispatches itself.
55      - Completion is a **bucket marker** (`<out_uri>/_dispatch/<token>.done`), not the Cloud
Run API status
56        ? so it reuses the storage layer + `execution_policy.poll_until` and survives a
control-plane restart.
57      - A **failed** Job currently writes **no** marker ? the dispatcher's bounded poll times
out
58        (`ExecutionPolicy.max_wait_sec`). A failure-marker is a follow-up.
59      - libx264 encode (parity, D16) + NVDEC decode-only; the image keeps ffmpeg's default
libx264.
60
```

14. assistant (2026-06-21T10:45:35.174Z)

Now let me examine the storage layer functions used (`fetch`, `parse_uri`, `get_storage`, `exists`, `put`) to assess the read-after-write race and the empty-marker handling precisely.

15. user (2026-06-21T10:45:35.979Z)

```
1       """Pluggable storage layer: local (default) + S3/GCS first-class buckets + Google Drive.
2
3       URI-driven (see ``ref.parse_uri``). The package import is cheap ? cloud SDKs are
imported
4       lazily, only when a non-local bucket URI is actually used (install the ``storage-s3`` /
5       ``storage-gcs`` extras). The default ``local`` backend needs no third-party deps and
preserves
6       today's behaviour byte-for-byte; ``gdrive://`` likewise needs none ? it reads a locally
MOUNTED
7       Google Drive (Google Drive for Desktop) as plain files.
8
9       Worker/dispatcher entry points speak URIs via the thin helpers ``fetch`` / ``put`` /
10      ``list_uris`` / ``get_storage``; ``local://`` (and bare paths) make ``fetch`` an
identity
11      passthrough. See ``docs/DECOUPLED_COMPUTE/``.
12      """
13      from __future__ import annotations
14
15      import os
16      import tempfile
17      from typing import Any, Dict, Iterator, Optional
18
19      from backend.storage.base import StorageBackend
20      from backend.storage.ref import StorageRef, StorageStat, parse_uri, resolve_input_uri
21
22      __all__ = [
23          "StorageBackend", "StorageRef", "StorageStat", "parse_uri", "resolve_input_uri",
24          "get_storage", "fetch", "put", "list_uris",
25          "acquire_local_input", "input_is_local", "resolve_input_ref",
26      ]
27
28
29      def _storage_dict(storage_config: Any) -> Dict[str, Any]:
30          """Coerce a StorageConfig (Pydantic) / dict / None into a plain settings dict."""
31          if storage_config is None:
32              return {}
33          if hasattr(storage_config, "model_dump"):
```

```
            return storage_config.model_dump()  # type: ignore[no-any-return]
        return dict(storage_config)


    def resolve_input_ref(raw: str, storage_config: Any = None) -> StorageRef:
        """Resolve a user-supplied input (a path or storage URI) into a ``StorageRef`` under
the input
        config (``input_backend``/``input_root``). Raises ``ValueError`` for an unsupported
URI scheme ?
        callers at the API/CLI boundary catch that and return a clean client error (never a
500).

        Use ``ref.backend`` to branch local-vs-remote and ``ref.key`` for the resolved LOCAL
path (e.g.
        ``local://?`` is stripped to the bare path), rather than stat-ing the raw URI
string."""
        sc = _storage_dict(storage_config)
        uri = resolve_input_uri(raw, input_backend=sc.get("input_backend", "local"),
                                input_root=sc.get("input_root", ""))
        return parse_uri(uri)


    def input_is_local(raw: str, storage_config: Any = None) -> bool:
        """True if ``raw`` resolves (under the input config) to a LOCAL filesystem path ?
i.e. a
        plain on-disk file the caller can stat/validate directly. False for a bucket / Drive
URI
        that must be fetched first. Raises ``ValueError`` for an unsupported scheme (same as
        ``resolve_input_ref``)."""
        return resolve_input_ref(raw, storage_config).backend == "local"


    def acquire_local_input(raw: str, storage_config: Any = None, *,
                            scratch_parent: "str | None" = None) -> str:
        """Resolve ``raw`` (a path or storage URI) to a LOCAL file path ready for
processing.

        Local / bare paths are returned UNCHANGED (identity passthrough, no copy ? today's
behaviour
        byte-for-byte). A bucket / Drive source (``gs://``/``s3://``/``gdrive://``, or a
bare name under a
        non-local ``input_root``/``input_backend``) is downloaded into a fresh scratch dir
and the local
        path returned. The scratch copy is left under the system temp dir (a pure
intermediate,
        regenerable from the source) and relies on OS temp reclamation ? the same pull
pattern the
        worker's ``cmd_infer`` already ships."""
        sc = _storage_dict(storage_config)
        ref = resolve_input_ref(raw, sc)
        if ref.backend == "local":
            return ref.key  # identity ? no temp, no copy, byte-identical to passing the
path directly
        scratch = tempfile.mkdtemp(prefix="rally-input-", dir=scratch_parent)
        return fetch(ref.uri, os.path.join(scratch, ref.name or "in.mp4"), config=sc)


    def _backend_class(name: str):
        """Lazy-resolve a backend class so cloud SDKs aren't imported until needed."""
        if name == "local":
            from backend.storage.local import LocalStorage
            return LocalStorage
        if name == "s3":
            from backend.storage.s3 import S3Storage
            return S3Storage
```

```
85          if name == "gcs":
86              from backend.storage.gcs import GCSStorage
87              return GCSStorage
88          if name == "gdrive":
89              from backend.storage.gdrive import GDriveStorage
90              return GDriveStorage
91          raise ValueError(f"unknown storage backend: {name!r} (expected
local|s3|gcs|gdrive)")
92
93
94      def get_storage(backend: Optional[str] = None, *,
95                      config: Optional[Dict[str, Any]] = None, **overrides) -> StorageBackend:
96          """Construct a backend. ``backend`` (or ``config['backend']``, default 'local')
selects
97          the implementation; its per-backend settings come from ``config[backend]`` merged
with
98          ``overrides``. Settings carry NO secrets ? only endpoints/regions/paths/source-
selectors."""
99          cfg = config or {}
100         name = backend or cfg.get("backend", "local")
101         settings: Dict[str, Any] = dict(cfg.get(name, {}) or {})
102         settings.update(overrides)
103         return _backend_class(name)(**settings)
104
105
106     def fetch(uri: str, dst: "str | os.PathLike[str] | None" = None, *,
107               config: Optional[Dict[str, Any]] = None, overwrite: bool = False) -> str:
108         """Resolve ``uri`` to a local file path. For ``local`` this is an identity
passthrough
109         (no copy); for s3/gcs it downloads to ``dst``."""
110         ref = parse_uri(uri)
111         return get_storage(ref.backend, config=config).fetch_to_local(ref, dst,
overwrite=overwrite)
112
113
114     def put(local_path: "str | os.PathLike[str]", uri: str, *,
115             config: Optional[Dict[str, Any]] = None, content_type: Optional[str] = None) ->
str:
116         """Upload/copy ``local_path`` to ``uri``; return ``uri``."""
117         ref = parse_uri(uri)
118         get_storage(ref.backend, config=config).put(local_path, ref,
content_type=content_type)
119         return uri
120
121
122     def list_uris(prefix_uri: str, *, config: Optional[Dict[str, Any]] = None) ->
Iterator[str]:
123         """Yield the URI of every object under ``prefix_uri``."""
124         ref = parse_uri(prefix_uri)
125         for r in get_storage(ref.backend, config=config).list(ref):
126             yield r.uri
127
```

16. assistant (2026-06-21T10:45:38.882Z)

Now let me look at the `fetch`/`exists`/`put` implementations for GCS specifically, since the read-after-write race and partial-read concerns depend on those. Let me check the GCS and local backends.

17. user (2026-06-21T10:45:39.847Z)

```
1       """Google Cloud Storage backend ? first-class object buckets.
2
3       Uses ``google-cloud-storage``, imported lazily so ``import backend.storage`` never pulls
it;
```

```
    install with the ``storage-gcs`` extra. Credentials come from Application Default Credentials
    (``GOOGLE_APPLICATION_CREDENTIALS`` pointing at a service-account JSON, or workload identity) ?
    never from config. A ``client`` may be injected for tests.
    """
    from __future__ import annotations

    from datetime import timedelta
    from typing import Any, Iterator, Optional

    from backend.storage.base import StorageBackend
    from backend.storage.ref import StorageRef, StorageStat


    class GCSStorage(StorageBackend):
        def __init__(self, *, client: Any = None, project: Optional[str] = None,
                     signed_url_ttl: int = 3600, **_ignored) -> None:
            self._ttl = signed_url_ttl
            if client is not None:
                self._client = client
                return
            from google.cloud import storage  # type: ignore  # lazy: only when a gcs:// URI is used
            self._client = storage.Client(project=project)

        def _blob(self, ref: StorageRef):
            return self._client.bucket(ref.bucket).blob(ref.key)

        def fetch_to_local(self, ref, dst=None, *, overwrite=False) -> str:
            if dst is None:
                raise ValueError("GCSStorage.fetch_to_local requires a dst path")
            import os
            dst = str(dst)
            if os.path.exists(dst) and not overwrite:
                return dst
            os.makedirs(os.path.dirname(dst) or ".", exist_ok=True)
            self._blob(ref).download_to_filename(dst)
            return dst

        def put(self, local_path, ref, *, content_type=None) -> StorageRef:
            self._blob(ref).upload_from_filename(str(local_path), content_type=content_type)
            return ref

        def exists(self, ref) -> bool:
            return bool(self._blob(ref).exists())

        def stat(self, ref) -> StorageStat:
            b = self._blob(ref)
            b.reload()
            return StorageStat(size=int(b.size or 0), modified=b.updated, etag=b.etag,
                               content_type=b.content_type)

        def list(self, prefix) -> Iterator[StorageRef]:
            for blob in self._client.list_blobs(prefix.bucket, prefix=prefix.key):
                yield StorageRef("gcs", prefix.bucket, blob.name,
f"gcs://{prefix.bucket}/{blob.name}")

        def delete(self, ref) -> None:
            self._blob(ref).delete()

        def signed_url(self, ref, *, ttl=3600, method="GET") -> str:
            return self._blob(ref).generate_signed_url(
                version="v4", expiration=timedelta(seconds=ttl or self._ttl), method=method)

```

18. user (2026-06-21T10:45:40.008Z)

```
1       """Local-filesystem storage backend ? the default, zero new dependencies.
2
3       Wraps today's behaviour byte-for-byte: ``fetch_to_local`` is an identity passthrough
(the
4       file is already local), ``put`` is a copy (a no-op when src == dst). With
``backend='local'``
5       the whole pipeline behaves exactly as before the storage layer existed.
6       """
7       from __future__ import annotations
8
9       import os
10      import shutil
11      from datetime import datetime
12      from typing import Iterator
13
14      from backend.storage.base import StorageBackend
15      from backend.storage.ref import StorageRef, StorageStat
16
17
18      class LocalStorage(StorageBackend):
19          def __init__(self, *, base_dir: str = "", **_ignored) -> None:
20              # Relative keys resolve under base_dir when set; absolute keys are used as-is.
21              self.base_dir = base_dir
22
23          def _path(self, ref: StorageRef) -> str:
24              p = ref.key
25              if self.base_dir and not os.path.isabs(p):
26                  return os.path.join(self.base_dir, p)
27              return p
28
29          def fetch_to_local(self, ref, dst=None, *, overwrite=False) -> str:
30              # The object is already on the local FS ? return its path (identity). ``dst`` is
31              # honoured only if it names a *different* location (a copy is then made).
32              src = self._path(ref)
33              if dst is None or os.path.abspath(str(dst)) == os.path.abspath(src):
34                  return src
35              dst = str(dst)
36              if os.path.exists(dst) and not overwrite:
37                  return dst
38              os.makedirs(os.path.dirname(dst) or ".", exist_ok=True)
39              shutil.copyfile(src, dst)
40              return dst
41
42          def put(self, local_path, ref, *, content_type=None) -> StorageRef:
43              dst = self._path(ref)
44              if os.path.abspath(str(local_path)) != os.path.abspath(dst):
45                  os.makedirs(os.path.dirname(dst) or ".", exist_ok=True)
46                  shutil.copyfile(str(local_path), dst)
47              return ref
48
49          def exists(self, ref) -> bool:
50              return os.path.exists(self._path(ref))
51
52          def stat(self, ref) -> StorageStat:
53              st = os.stat(self._path(ref))
54              return StorageStat(size=st.st_size,
modified=datetime.fromtimestamp(st.st_mtime))
55
56          def list(self, prefix) -> Iterator[StorageRef]:
57              base = self._path(prefix)
58              if os.path.isdir(base):
59                  for root, _dirs, files in os.walk(base):
60                      for fn in sorted(files):
```

```
61                          full = os.path.join(root, fn)
62                          yield StorageRef("local", "", full, f"local://{full}")
63              elif os.path.exists(base):
64                  yield StorageRef("local", "", base, f"local://{base}")
65
66          def delete(self, ref) -> None:
67              path = self._path(ref)
68              if os.path.exists(path):
69                  os.remove(path)
70
```

19. assistant (2026-06-21T10:45:45.136Z)

Now I have a complete picture. Let me analyze the key concern about `fetch` caching in the read-after-write race scenario. There's a subtle but important issue: `fetch_to_local` for GCS returns the cached `dst` if it already exists (line 35-36: `if os.path.exists(dst) and not overwrite: return dst`). Let me trace the `_read_json` path carefully.

In `CloudRunCompute._read_json`, it calls `fetch(uri, config=self._storage_config)` with **no `dst` argument**. For GCS that raises `ValueError("GCSStorage.fetch_to_local requires a dst path")` ? caught by the bare `except Exception` returning `{}`. Let me verify this is a real problem by checking how `fetch` is called with no dst against GCS.

20. user (2026-06-21T10:45:46.681Z)

```
backend\storage\base.py:22:    def fetch_to_local(self, ref: StorageRef, dst: "str |
os.PathLike[str] | None" = None,
backend\storage\base.py-23-                       *, overwrite: bool = False) -> str:
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\storage\base.py-24-        """Make
the object available as a local file; return its local path. For the
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\storage\base.py-25-        ``local``
backend this is an identity passthrough (the object is already local)."""
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\storage\base.py-26-
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\storage\base.py-27-
@abstractmethod
backend\storage\base.py-28-    def put(self, local_path: "str | os.PathLike[str]", ref:
StorageRef, *,
backend\storage\base.py-29-            content_type: "str | None" = None) -> StorageRef:
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\storage\base.py-30-
"""Upload/copy a local file to ``ref``; return ``ref``."""
--
backend\storage\gcs.py:30:    def fetch_to_local(self, ref, dst=None, *, overwrite=False) ->
str:
backend\storage\gcs.py-31-        if dst is None:
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\storage\gcs.py-32-            raise
ValueError("GCSStorage.fetch_to_local requires a dst path")
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\storage\gcs.py-33-        import os
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\storage\gcs.py-34-        dst =
str(dst)
backend\storage\gcs.py-35-        if os.path.exists(dst) and not overwrite:
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\storage\gcs.py-36-            return
dst
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\storage\gcs.py-37-
os.makedirs(os.path.dirname(dst) or ".", exist_ok=True)
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\storage\gcs.py-38-
self._blob(ref).download_to_filename(dst)
--
backend\storage\gdrive.py:112:    def fetch_to_local(self, ref, dst=None, *, overwrite=False) ->
str:
backend\storage\gdrive.py-113-        # The file is already local (via the mount): no dst =>
identity passthrough to the mount
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\storage\gdrive.py-114-        #
path; a dst => copy out (a byte-stable scratch copy), exactly as the local backend does.
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\storage\gdrive.py-115-
self._require_present(ref)
```

```
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\storage\gdrive.py-116-        return
self._local.fetch_to_local(self._local_ref(ref), dst, overwrite=overwrite)
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\storage\gdrive.py-117-
backend\storage\gdrive.py-118-    def put(self, local_path, ref, *, content_type=None) ->
StorageRef:
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\storage\gdrive.py-119-
self._local.put(local_path, self._local_ref(ref), content_type=content_type)
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\storage\gdrive.py-120-        return
ref
--
backend\storage\local.py:29:    def fetch_to_local(self, ref, dst=None, *, overwrite=False) ->
str:
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\storage\local.py-30-        # The
object is already on the local FS ? return its path (identity). ``dst`` is
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\storage\local.py-31-        #
honoured only if it names a *different* location (a copy is then made).
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\storage\local.py-32-        src =
self._path(ref)
backend\storage\local.py-33-        if dst is None or os.path.abspath(str(dst)) ==
os.path.abspath(src):
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\storage\local.py-34-
return src
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\storage\local.py-35-        dst =
str(dst)
backend\storage\local.py-36-        if os.path.exists(dst) and not overwrite:
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\storage\local.py-37-
return dst
--
backend\storage\s3.py:38:    def fetch_to_local(self, ref, dst=None, *, overwrite=False) -> str:
backend\storage\s3.py-39-        if dst is None:
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\storage\s3.py-40-            raise
ValueError("S3Storage.fetch_to_local requires a dst path")
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\storage\s3.py-41-        dst =
str(dst)
backend\storage\s3.py-42-        if os.path.exists(dst) and not overwrite:
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\storage\s3.py-43-            return
dst
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\storage\s3.py-44-
os.makedirs(os.path.dirname(dst) or ".", exist_ok=True)
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\storage\s3.py-45-
self._client.download_file(ref.bucket, ref.key, dst)
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\storage\s3.py-46-        return dst
```

21. user (2026-06-21T10:45:47.827Z)

```
1       """S3 (and S3-compatible) storage backend ? first-class object buckets.
2
3       One backend covers **AWS S3, Cloudflare R2, DigitalOcean Spaces, and MinIO** ? they
differ
4       only by ``endpoint_url`` (+ region / addressing style). Uses ``boto3``, imported lazily
so
5       ``import backend.storage`` never pulls it; install with the ``storage-s3`` extra.
6
7       Credentials are NEVER read from config ? only ``endpoint_url`` / ``region`` / addressing
8       style. boto3's default chain supplies creds (``AWS_ACCESS_KEY_ID`` /
``AWS_SECRET_ACCESS_KEY``
9       / ``AWS_SESSION_TOKEN`` env, instance role, or ``~/.aws``). A ``client`` may be injected
10      (tests pass a fake; production lets the constructor build one via boto3).
11      """
12      from __future__ import annotations
13
14      import os
15      from typing import Any, Iterator, Optional
16
17      from backend.storage.base import StorageBackend
```

```
18      from backend.storage.ref import StorageRef, StorageStat
19
20
21      class S3Storage(StorageBackend):
22          def __init__(self, *, client: Any = None, endpoint_url: Optional[str] = None,
23                       region: Optional[str] = None, addressing_style: str = "auto",
24                       signed_url_ttl: int = 3600, **_ignored) -> None:
25              self._ttl = signed_url_ttl
26              if client is not None:
27                  self._client = client
28                  return
29              import boto3  # type: ignore  # lazy: only when an s3:// URI is actually used
30              from botocore.config import Config  # type: ignore
31              self._client = boto3.client(
32                  "s3",
33                  endpoint_url=endpoint_url or os.environ.get("AWS_ENDPOINT_URL_S3") or None,
34                  region_name=region,
35                  config=Config(s3={"addressing_style": addressing_style}),
36              )
37
38          def fetch_to_local(self, ref, dst=None, *, overwrite=False) -> str:
39              if dst is None:
40                  raise ValueError("S3Storage.fetch_to_local requires a dst path")
41              dst = str(dst)
42              if os.path.exists(dst) and not overwrite:
43                  return dst
44              os.makedirs(os.path.dirname(dst) or ".", exist_ok=True)
45              self._client.download_file(ref.bucket, ref.key, dst)
46              return dst
47
48          def put(self, local_path, ref, *, content_type=None) -> StorageRef:
49              extra = {"ContentType": content_type} if content_type else None
50              self._client.upload_file(str(local_path), ref.bucket, ref.key, ExtraArgs=extra)
51              return ref
52
53          def exists(self, ref) -> bool:
54              try:
55                  self._client.head_object(Bucket=ref.bucket, Key=ref.key)
56                  return True
57              except Exception:  # noqa: BLE001 - any boto/client error means "not found /
unreachable"
58                  return False
59
60          def stat(self, ref) -> StorageStat:
```

22. assistant (2026-06-21T10:45:56.740Z)

This confirms a real, significant bug. `CloudRunCompute._read_json` calls `fetch(uri, config=...)` with **no `dst`**. For both GCS and S3 backends, `fetch_to_local` with `dst=None` **raises `ValueError`** ("requires a dst path"). Since `_read_json` wraps everything in `except Exception: return {}`, this means:

For the real cloud path (gcs:// / s3://), `_read_marker` will ALWAYS return `{}` after `exists()` is true (because `_read_json(done_uri)` raises ValueError ? caught ? returns `{}`). And `_read_json` for `report.json` will likewise always return `{}`. So `video_id` is always `""`, `outputs_uri` is always `""`, report always `{}` ? the result is `ComputeResult(returncode=0, video_id="", outputs_uri="", report={})`. The cloud path is fundamentally broken for the real backends; the tests only pass because they use the `local` backend where `fetch` with `dst=None` is an identity passthrough.

This is the most important finding. Let me also verify the `parse_uri` for gs:// scheme and the `_FAST` policy `poll_every_n_sec=0.0` edge. Let me check `parse_uri`.

23. user (2026-06-21T10:45:57.582Z)

```
"""Storage value objects + URI parsing for the pluggable storage layer.

A ``StorageRef`` is a backend-agnostic pointer to one object. URIs:

    <bare path>                 -> local   (today's behaviour; e.g. C:/v/x.mp4, /data/x.mp4)
    local://<path>              -> local   (local:///abs or local://rel/p)
    s3://<bucket>/<key>         -> s3       (AWS + R2 + DO Spaces + MinIO via endpoint_url)
    gcs://<bucket>/<key>        -> gcs      (gs:// accepted, normalized to gcs://)
    gdrive://<path>             -> gdrive   (path under your locally-MOUNTED Google Drive "My Drive")

No cloud SDKs are imported here ? this module is pure stdlib so ``import backend.storage``
stays cheap.
"""
from __future__ import annotations

import re
from dataclasses import dataclass
from datetime import datetime
from typing import Optional

_SCHEME_RE = re.compile(r"^([A-Za-z][A-Za-z0-9+.\-]*)://")
_KNOWN = ("local", "s3", "gcs", "gdrive")


@dataclass(frozen=True)
class StorageStat:
    """Lightweight object metadata (uniform across backends)."""
    size: int
    modified: Optional[datetime] = None
    etag: Optional[str] = None
    content_type: Optional[str] = None


@dataclass(frozen=True)
class StorageRef:
    """A backend + a location. ``bucket`` is the S3/GCS bucket; for ``local`` and ``gdrive`` it is
    empty and ``key`` carries the path (a filesystem path, or a path under the mounted Drive)."""
    backend: str
    bucket: str
    key: str
    uri: str

    @property
    def name(self) -> str:
        return self.key.rstrip("/").rsplit("/", 1)[-1]


def parse_uri(uri: str) -> StorageRef:
    """Parse a storage URI (or a bare filesystem path) into a ``StorageRef``.

    A string with no ``scheme://`` prefix (including Windows ``C:\\...`` paths, which
    have no ``//``) is treated as a local filesystem path ? preserving today's behaviour.
    """
    s = str(uri)
    m = _SCHEME_RE.match(s)
    if not m:
        return StorageRef("local", "", s, f"local://{s}")
    scheme = m.group(1).lower()
```

```
59          rest = s[m.end():]
60          if scheme == "gs":
61              scheme = "gcs"
62          if scheme in ("local", "gdrive"):
63              # Path-keyed, no bucket: local FS, or a path under the mounted Google Drive "My
Drive".
64              return StorageRef(scheme, "", rest, s)
65          if scheme in _KNOWN:
66              bucket, _, key = rest.partition("/")
67              return StorageRef(scheme, bucket, key, s)
68          raise ValueError(f"unsupported storage URI scheme: {scheme}:// (in {s!r})")
69
70
71      def resolve_input_uri(raw: str, *, input_backend: str = "local", input_root: str = "")
-> str:
72          """Resolve a user-supplied input reference into a storage URI
(COMPUTE_DECOUPLED_SERVING M2).
73
74          Precedence (first match wins):
75            1. An explicit ``scheme://`` on ``raw`` (``gs://``, ``s3://``, ``local://``,
``gdrive://``) ? as-is.
76            2. An ABSOLUTE local path (``/data/x.mp4``, ``C:\\?``, ``C:/?``) ? as-is (always
local).
77            3. A bare/relative name + a non-empty ``input_root`` ? joined under it
78               (``"gs://bucket/videos"`` + ``"x.mp4"`` ? ``"gs://bucket/videos/x.mp4"``).
79            4. A bare/relative name + ``input_backend != "local"`` ? prefixed (``"gcs"`` +
``"bucket/x"`` ?
80               ``"gcs://bucket/x"``).
81            5. Otherwise ? ``raw`` unchanged (a relative/bare LOCAL path ? today's behaviour).
82
83          So the default (``input_backend="local"``, ``input_root=""``) is an identity
passthrough for every
84          path used today, preserving byte-for-byte behaviour.
85          """
86          s = str(raw)
87          if _SCHEME_RE.match(s):           # 1. explicit scheme wins
88              return s
89          # 2. an anchored LOCAL path is always local, never joined. Detect this OS-
INDEPENDENTLY ?
90          #    os.path.isabs is platform- AND Python-version-dependent (on Windows + py3.13 a
POSIX-style
91          #    "/data/x" is NOT absolute). Anchored == a leading "/" or "\\", or a Windows
drive prefix "X:".
92          if s.startswith(("/", "\\")) or (len(s) >= 2 and s[1] == ":" and s[0].isalpha()):
93              return s
94          if input_root:                    # 3. join a relative name under the configured
root
95              return input_root.rstrip("/") + "/" + s.lstrip("/")
96          if input_backend and input_backend != "local":   # 4. prefix a bucket/key with the
declared backend
97              return f"{input_backend}://{s}"
98          return s                          # 5. bare/relative local path ? unchanged
99
```

24. assistant (2026-06-21T10:46:06.321Z)

Confirmed. Now let me double-check the `_read_marker` flow for the read-after-write race more precisely, and verify the worker writes the marker LAST (after pushing outputs). Looking at `cmd_infer`:

- Line 226-237: serialize + push outputs (report.json, intervals.csv) to `outputs/<video_id>/`
- Line 247-248: write done-marker LAST

So ordering is correct: outputs pushed before marker. Good. But the read-after-write race on the marker body itself: `_read_marker` does `exists(ref)` then `_read_json(done_uri)`. On GCS, `put`

(upload_from_filename) is atomic ? the object becomes visible only once fully uploaded, so there's no partial-body read for GCS. The marker is also tiny. So for GCS the partial-read concern is largely mitigated by object-store atomicity. The bigger issue is the `dst=None` ValueError I found, which makes the read fail entirely on cloud backends.

Let me also verify the test for the empty-marker handling and confirm `_read_json` returns `{}` on empty file (`json.load` on empty file raises ? caught ? `{}`). And confirm the `report.json` filename concatenation: `outputs_uri` ends with `/`, so `f"{outputs_uri}report.json"` is correct.

Let me run the existing tests to confirm they pass (the local-backend path) and confirm my reading that cloud is broken is not contradicted by some `dst`-injection I missed.

25. user (2026-06-21T10:46:16.414Z)

```
.........                                                                [100%]
9 passed in 6.07s
```

26. assistant (2026-06-21T10:46:27.973Z)

Tests pass on the local backend. Now let me confirm the `_read_json` ValueError-on-cloud theory by checking whether `fetch` is ever given a `dst` anywhere in `cloud_run.py`. It is not ? both `_read_marker`?`_read_json(done_uri)` and `_read_json(f"{outputs_uri}report.json")` call `fetch(uri, config=...)` with no `dst`. This is definitively broken for gcs/s3.

Let me verify one more subtle robustness point: the `GoogleCloudRunJobClient.run_job` return-value expression on line 77, which has operator-precedence subtlety, and check whether `job_path` requires a non-None project (it does ? `client.job_path(project, region, job_name)` with `project=None` from env when `GOOGLE_CLOUD_PROJECT` unset would produce a malformed path). Let me check the `__init__.py` default: `project = os.environ.get("GOOGLE_CLOUD_PROJECT")` ? `None` if unset. That would be passed to `job_path(None, ...)`. This is a real edge but owner-verifies on M7; I'll note it as minor.

Let me confirm line 77's logic: `getattr(op, "metadata", None) and getattr(op.metadata, "name", "") or "execution"`. If `op.metadata` is None ? `None and ... or "execution"` ? `"execution"`. If metadata exists but name is `""` ? `... and "" or "execution"` ? `"execution"`. If name is non-empty ? returns it. This is only used for logging, so it's benign. Fine.

Now let me assess the failure-mode #2 (PolicyTimeout propagation). `poll_until` raises `PolicyTimeout` ? propagates out of `run_infer` ? out of `_dispatch_remote` (no catch) ? out of `cmd_infer` ? out of `main` ? uncaught traceback, `sys.exit` never reached cleanly, process exits with traceback (rc 1). The README explicitly documents this as accepted behavior ("a failure-marker is a follow-up"). I'll note it as minor/by-design since it produces a noisy traceback rather than a clean rc, which contradicts the worker's otherwise-clean rc contract (0/2/3).

I have enough to report. Let me compose the findings.

27. user (2026-06-21T10:46:50.896Z)

Structured output provided successfully

28. assistant (2026-06-21T10:47:03.441Z)

I've completed the cloud-protocol robustness review of the M6 change. Summary of what I found:

Blocker ? `backend/compute/cloud_run.py:127-141`: The bucket-poll is broken for the actual cloud backends it targets. `_read_json` calls `fetch(uri, config=...)` with **no `dst`**, but `GCSStorage`/`S3Storage.fetch_to_local` raise `ValueError` when `dst` is None (gcs.py:31-32, s3.py:39-40). That `ValueError` is swallowed by `_read_json`'s bare `except Exception: return {}`, so on the real gcs/s3 path the done-marker read AND the report.json read always return `{}`. Every cloud job therefore returns `ComputeResult(rc=0, video_id="", outputs_uri="", report={})` regardless of what the worker did. The tests pass only because they all use the `local` backend, where `fetch(uri, dst=None)` is an identity passthrough ? so the cloud read

path has zero real coverage.

Minor (2):
- A present-but-garbled/empty marker is treated as rc=0 success (`marker.get("returncode", 0)`, line 122) with `video_id=""` ? a cloud job that genuinely failed can be reported as a clean success.
- A failed/timed-out container surfaces as an uncaught `PolicyTimeout` traceback out of `_dispatch_remote`/`cmd_infer` (rc 1), rather than the worker's documented clean rc contract. Catching it in `_dispatch_remote` and returning a logged non-zero rc would fix the surfacing even before the planned failure-marker work.

Checked and found sound: the write/read ordering is safe (worker pushes outputs at __main__.py:226-237, then writes the marker LAST at 247-248); GCS `put` atomicity means the tiny marker body isn't visible partially-flushed; the empty-marker `{}`-terminates-poll choice is intentional; and the `google-cloud-run` v2 usage (`JobsClient` / `job_path` / `RunJobRequest.Overrides.ContainerOverride(args=...)` / `run_job`) is plausible ? the line 77 return expression is only used for logging and degrades to `"execution"` safely. The one real edge there (`project=None` from an unset `GOOGLE_CLOUD_PROJECT` passed to `job_path`) is owner-verified on M7, so I did not raise it.